

Chapter5 Notes

Unit V: Advance Topics on CSS

[7 Hrs.]

1. CSS Flexbox

- 1-Dimensional layout system — arranges items in a **row** or **column**.
- Real project use: navbars, button groups, card rows, centering content.

```
.container {
  display: flex;
  flex-direction: row;          /* row (default) | column | row-reverse | column-
    reverse */
  justify-content: space-between; /* main-axis alignment: flex-start, center, space-
    around, space-between */
  align-items: center;          /* cross-axis alignment: flex-start, center, stretch
    */
  flex-wrap: wrap;              /* allows items to wrap to next line on small screens
    */
  gap: 20px;                    /* spacing between items - replaces margin hacks */
}

.item {
  flex: 1;                      /* item grows/shrinks equally to fill space */
}
```

Real project example — e-commerce navbar with logo, links, and cart icon spread apart:

```
<header class="navbar">
  <div class="logo">ShopEasy</div>
  <nav class="nav-links">
    <a href="#">Home</a>
    <a href="#">Shop</a>
    <a href="#">Contact</a>
  </nav>
  <div class="cart-icon"><img alt="cart icon" data-bbox="338 848 358 860"/> Cart (2)</div>
</header>
```

```
.navbar {
  display: flex;
```

```

    justify-content: space-between;
    align-items: center;
    padding: 15px 30px;
    background: #1e293b;
}
.nav-links { display: flex; gap: 20px; }

```

Real project example — pricing cards row that wraps on mobile:

```

.pricing-container {
    display: flex;
    justify-content: center;
    flex-wrap: wrap;
    gap: 24px;
}
.pricing-card { flex: 1 1 250px; } /* grow, shrink, base width 250px */

```

2. CSS Grid

- 2-Dimensional layout system — arranges items in **rows AND columns** simultaneously.
- Real project use: product galleries, dashboards, image galleries, magazine-style layouts.

```

.grid-container {
    display: grid;
    grid-template-columns: repeat(3, 1fr); /* 3 equal columns */
    grid-template-rows: auto;
    gap: 20px;
}
.featured {
    grid-column: span 2; /* item spans 2 columns - hero product */
}

```

Real project example — e-commerce product gallery (auto-responsive, no media query needed):

```

.product-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
    gap: 20px;
}

```

- `auto-fit` + `minmax()` — cards automatically reflow from 4 → 3 → 2 → 1 columns as screen shrinks, without writing separate media queries. Common modern real-project trick.

Real project example — dashboard layout with named areas:

```
.dashboard {  
  display: grid;  
  grid-template-columns: 250px 1fr;  
  grid-template-areas:  
    "sidebar header"  
    "sidebar main";  
  height: 100vh;  
}  
.sidebar { grid-area: sidebar; background: #1e293b; }  
.header { grid-area: header; }  
.main { grid-area: main; padding: 20px; }
```

```
<div class="dashboard">  
  <aside class="sidebar">Menu</aside>  
  <header class="header">Dashboard</header>  
  <main class="main">Widgets here</main>  
</div>
```

Flexbox vs Grid: | Flexbox | Grid | — | — | 1D (row OR column) | 2D (rows AND columns) | |
Content-driven sizing | Layout-driven (define structure first) | | Navbars, button groups | Full page layouts, galleries, dashboards |

3. CSS Transitions and Animations

Transitions — smooth change between two states (e.g., hover)

```
.btn {  
  background: #2563eb;  
  transition: background 0.3s ease, transform 0.3s ease;  
}  
.btn:hover {  
  background: #1d4ed8;  
  transform: scale(1.05);  
}
```

Real project use: button hover effects, image zoom on hover (product cards), smooth color/size change — used on nearly every modern website.

Animations — multi-step, can run automatically/loop, defined with `@keyframes`

```

@keyframes fadeIn {
  from { opacity: 0; transform: translateY(20px); }
  to   { opacity: 1; transform: translateY(0); }
}
.hero-text {
  animation: fadeIn 1s ease-in-out;
}

@keyframes spin {
  from { transform: rotate(0deg); }
  to   { transform: rotate(360deg); }
}
.loading-spinner {
  animation: spin 1s linear infinite;
}

```

Real project use: page load fade-in effects (landing pages), loading spinners (before data loads from server), notification “slide-in” toast messages.

```

/* Real project: toast notification sliding in from right */
@keyframes slideIn {
  from { right: -300px; opacity: 0; }
  to   { right: 20px; opacity: 1; }
}
.toast {
  position: fixed;
  top: 20px;
  animation: slideIn 0.4s ease-out forwards;
}

```

4. Responsive Web Design (Advanced)

Building on Unit III basics — **mobile-first approach** used in most real modern projects.

Mobile-first: write base styles for mobile, then override for larger screens using `min-width`

```

/* Base (mobile) styles */
.container { padding: 10px; }
.product-grid { grid-template-columns: 1fr; }

/* Tablet and up */
@media (min-width: 768px) {
  .container { padding: 20px; }
}

```

```

.product-grid { grid-template-columns: repeat(2, 1fr); }

/* Desktop and up */
@media (min-width: 1024px) {
  .container { padding: 40px; }
  .product-grid { grid-template-columns: repeat(4, 1fr); }
}

```

Why real projects prefer mobile-first: majority of web traffic is mobile; writing base CSS for mobile keeps default page light/fast, then progressively enhances for bigger screens.

Common breakpoints used in industry

Device	Width
Mobile	up to 480px
Tablet	481px – 768px
Laptop	769px – 1024px
Desktop	1025px+

5. CSS Specificity and Inheritance

Specificity — determines which rule “wins” when multiple rules target same element

Order (lowest to highest): 1. Element selectors (`p`, `div`) — weight 0-0-1 2. Class, attribute, pseudo-class (`.btn`, `[type=text]`, `:hover`) — weight 0-1-0 3. ID selectors (`#header`) — weight 1-0-0 4. Inline style (`style=""`) — overrides all 5. `!important` — overrides everything (avoid overusing — real teams flag this in code review)

```

p { color: blue; } /* specificity: 0-0-1 */
.text { color: green; } /* specificity: 0-1-0 - wins over p */
#main-text { color: red; } /* specificity: 1-0-0 - wins over .text */

```

```

<p id="main-text" class="text">This text will be RED</p>

```

Real project bug this explains: developer writes `.btn { color: white; }` but button still shows black — because a more specific rule like `#header .btn { color: black; }` exists elsewhere and wins. Understanding specificity is essential for debugging CSS conflicts in large codebases.

Inheritance — some properties automatically pass from parent to child

```
body {  
  font-family: Arial, sans-serif; /* inherited by all text inside body */  
  color: #333;  
}
```

- **Inherited properties:** color, font-family, font-size, line-height, text-align.
- **Non-inherited properties:** margin, padding, border, width, height, background (must set explicitly on each element).

6. CSS Units and Values

Unit	Type	Relative to	Real use
px	Absolute	Fixed pixel	Borders, precise small elements
%	Relative	Parent element	Fluid widths (width: 50%)
em	Relative	Parent's font-size	Spacing that scales with text
rem	Relative	Root (html) font-size	Preferred for consistent sizing across whole site
vw / vh	Relative	Viewport width/height	Full-screen hero sections (height: 100vh)
vmin / vmax	Relative	Smaller/larger of vw or vh	Responsive font scaling

Real project example — using rem for consistent, scalable typography:

```
html { font-size: 16px; } /* 1rem = 16px */  
h1 { font-size: 2.5rem; } /* 40px, but scales if user changes browser font size  
    (accessibility) */  
p { font-size: 1rem; } /* 16px */  
.card { padding: 1.5rem; }
```

Why `rem` preferred in real projects over `px` : respects user's browser accessibility settings (font-size zoom), and changing root font-size instantly rescales entire site proportionally.

```
/* Real project: full-screen hero section */
.hero {
  height: 100vh;
  width: 100vw;
}
```

7. CSS Preprocessors (SASS/SCSS overview)

- Preprocessors add programming features (variables, nesting, mixins, functions) to CSS, then **compile down to normal CSS**.
- Most popular: **SASS/SCSS**. Used in large real projects (design systems, big company sites) for maintainability.

Variables

```
$primary-color: #2563eb;
$spacing: 16px;

.btn {
  background: $primary-color;
  padding: $spacing;
}
```

Nesting

```
.navbar {
  background: #1e293b;
  ul {
    display: flex;
    li {
      margin-right: 15px;
      a { color: white; text-decoration: none; }
    }
  }
}
```

Compiles to:

```
.navbar { background: #1e293b; }
.navbar ul { display: flex; }
```

```
.navbar ul li { margin-right: 15px; }  
.navbar ul li a { color: white; text-decoration: none; }
```

Mixins (reusable style blocks)

```
@mixin flex-center {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}  
  
.modal-overlay {  
  @include flex-center;  
  height: 100vh;  
}
```

Functions

```
@function rem($px) {  
  @return $px / 16px * 1rem;  
}  
h1 { font-size: rem(32px); } /* outputs 2rem */
```

Real project use: design systems define color palette + spacing scale once as SCSS variables, reused across hundreds of components — change brand color in one place, updates entire site.

8. CSS Best Practices and Optimization

1. **Use external CSS files**, organized by component/page (e.g., `navbar.css`, `footer.css`) or single `styles.css` with clear sections.
2. **Use `rem` / % over fixed `px`** for scalable, accessible layouts.
3. **Avoid `!important`** — fix specificity properly instead; overuse makes debugging painful in real teams.
4. **Use CSS variables (custom properties)** for theming:

```
:root {  
  --primary-color: #2563eb;  
  --spacing-md: 16px;  
}  
  
.btn {  
  background: var(--primary-color);  
  padding: var(--spacing-md);  
}
```


Real use: dark mode toggle — just swap `:root` variable values via JS, entire site restyles instantly.

5. **Minify CSS** for production (removes whitespace/comments) — smaller file, faster load.

6. **Use vendor prefixes** where needed for older browser support:

```
.box {  
  display: -webkit-flex;    /* older Safari */  
  display: flex;  
}
```

(Tools like Autoprefixer handle this automatically in real projects.) 7. **Avoid deep nesting** in SCSS (max 3 levels) — deeply nested selectors hurt performance and readability. 8. **Group related properties** logically (positioning, box model, typography, visual) for readability. 9. **Remove unused CSS** (tools like PurgeCSS) — reduces file size in production builds. 10. **Mobile-first + `auto-fit / minmax()` in Grid** — write less CSS to achieve full responsiveness.

Quick Recap (Exam Points)

- Flexbox = 1D layout (`justify-content`, `align-items`, `flex-wrap`); Grid = 2D layout (`grid-template-columns`, `grid-template-areas`).
- Transitions = smooth state change (hover); Animations = multi-step via `@keyframes`, can loop (`infinite`).
- Mobile-first responsive design = base styles for mobile, `min-width` media queries scale up.
- Specificity order: inline > ID > class/attribute/pseudo-class > element; `!important` overrides all (avoid).
- Inherited properties: color, font-family, font-size; Non-inherited: margin, padding, border, width.
- Units: `px` (absolute), `% / em / rem` (relative), `vw / vh` (viewport-relative). `rem` preferred for accessibility.
- Preprocessors (SASS/SCSS): variables (`$var`), nesting, mixins (`@mixin / @include`), functions — compile to plain CSS.
- Best practices: CSS variables for theming, avoid `!important`, minify for production, mobile-first, avoid deep nesting.

Possible Exam Questions

Short Answer / Conceptual

1. Differentiate between CSS Flexbox and CSS Grid with examples.
2. What is the difference between CSS transitions and CSS animations?
3. What is mobile-first responsive design? Why is it preferred in real projects?
4. Explain CSS specificity with an example showing which rule wins.
5. What is the difference between inherited and non-inherited CSS properties? Give two examples of each.
6. Differentiate between `px`, `em`, and `rem` units.
7. What is a CSS preprocessor? Name two features SASS provides over plain CSS.
8. What are CSS custom properties (variables)? How are they used for theming/dark mode?
9. Why should `!important` be avoided in real-world CSS codebases?
10. What is the use of `minmax()` and `auto-fit` in CSS Grid?

Long Answer / Programming

1. Design a responsive product gallery using CSS Grid with `auto-fit / minmax()` that works without media queries.
2. Write CSS to create a button with a smooth hover transition (color change + scale effect).
3. Create a `@keyframes` animation for a loading spinner and a fade-in effect for page content.
4. Write SCSS code using variables, nesting, and a mixin to style a navigation bar, and show the compiled CSS output.
5. Design a dashboard layout (sidebar + header + main content) using CSS Grid template areas.
6. Explain and demonstrate mobile-first design by writing CSS for a 3-column layout that becomes 1-column on mobile using `min-width` media queries.
7. Implement a dark mode toggle using CSS custom properties (`:root` variables) and JavaScript.

True/False or Fill in the Blanks

1. CSS Grid is a two-dimensional layout system. (True)
2. `em` unit is relative to the root element's font-size. (False — that's `rem`)
3. `!important` should be used frequently to fix styling issues. (False)
4. _____ property in Flexbox controls alignment along the main axis. (`justify-content`)
5. SCSS variables are declared using the _____ symbol. (`$`)
6. _____ rule is used to define animation steps in CSS. (`@keyframes`)